

Code-Layout, Readability and Reusability

Date	04 July 2026
Team ID	XXXXXX
Project Name	Emotion Support Engine
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code.	Yes	Python strictly enforces 4-space indentation, ensuring perfectly readable blocks.
2	Meaningful Naming Conventions	Variables and functions have descriptive, clear names.	Yes	Used clear variable names like <code>user_input</code> and <code>chat_session</code> instead of random letters.
3	Comments and Documentation	Code includes necessary inline comments and docstrings.	Yes	Added comments to explain the Streamlit UI setup and the Gemini API routing.
4	Modular Design	Code is broken down into reusable functions/components.	Yes	Separated the web interface design (<code>st.set_page_config</code>) from the AI generation logic.
5	Error Handling	Proper use of try/except blocks or fallback logic.	Yes	Included error handling for the API connection to prevent app crashes if Google's servers are busy.

6	No Hardcoded Secrets	API keys and passwords are kept out of the main code.	Yes	Utilized <code>st.secrets["API_KEY"]</code> to securely store the Google Gemini key outside the codebase
---	----------------------	---	-----	--

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
1	Page UI Configuration (setup_page)	Python / Streamlit	Reused at the start of every session to render the web interface layout and styling.	High

2	API Client Initialization (init_gemini)	Python / Gemini API	Reused whenever the app needs to establish a secure connection to Google's AI servers.	High
3	Session State Manager	Python / Streamlit	Reused for every single user message to append text to the active chat history.	High
4	Prompt Formatter (format_input)	Python	Reused to structure and inject the user's emotions into the prompt before sending it to the AI.	Medium
5	Error / Exception Handler	Python	Reused across the app to gracefully catch API timeouts or empty user inputs without crashing.	Medium

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5	The codebase is highly organized, with a clear separation between the frontend (Streamlit) UI and the backend AI logic.
Readability	5	Uses standard 4-space Python indentation, clear variable names, and avoids complex, nested loops.
Reusability	4	Core functions (like API routing and session management) are modular and could easily be reused in future projects.
Documentation / Comments	4	Important functions and API logic are well-documented with inline comments for easy team collaboration.
Overall Score	5	API keys are securely hidden using <code>st.secrets</code> , and <code>try-except</code> blocks prevent the app from crashing on errors.